
```
function getNewPuzzle() {
    // Get a new puzzle by making random slides from the solved state.
    let board = getNewBoard();
    for (let i = 0; i < DIFFICULTY; i++) {
        let validMoves = getValidMoves(board);
        makeMove(board, validMoves[Math.floor(Math.random() * validMoves.length)]);
    }
    return board;
}
```

The call to `getNewBoard()` obtains a board data structure in the ordered, solved state. The `for` loop calls `getValidMoves()` to obtain a list of valid moves, given the current state of the board, and then calls `makeMove()` with a randomly selected move from the list. The `random.choice()` function in Python and the `Math.floor()` and `Math.random()` functions in JavaScript will handle the random selection from the `validMoves` list or array, no matter what combination of `UP`, `DOWN`, `LEFT`, and `RIGHT` values it contains.

The `DIFFICULTY` constant determines how many random slides from `makeMove()` the `for` loop applies. The higher the integer in `DIFFICULTY`, the more scrambled the puzzle becomes. Even though this results in some moves that undo earlier moves by pure chance, such as sliding left and then immediately sliding right, with enough slides the function produces a thoroughly scrambled board. For testing purposes, `DIFFICULTY` is set to 40, allowing the program to produce a solution in about a minute. For a more realistic 15-puzzle, you should change `DIFFICULTY` to 200.

After the board data structure in `board` is created and scrambled, the `getNewPuzzle()` function returns it.